



記事タイトル

「agent は、知識を足せば足すほど賢くなる」

素直に考えると、そうなるはずです。docs を書き、CLAUDE.md に規約を足し、skill を整備する。

実際、僕もみなさんと同じようにハーネスを育ててきました。

ところが、ある種の失敗だけは いつまでも消えません

書いてあるのに、読まない。

docs は手元にある。手順も注意書きも、agent が読める場所にちゃんと置いてある。それでも肝心の瞬間になると、agent はそれを見ずに別のやり方で走り出す。知識を足しても直らない。むしろ足すほど、常時読ませる文章が context を圧迫していく。で、結局どうすればいいんだ？と。

しばらく付き合ってみて、見立てが変わりました

人間の記憶で言えば、「覚えている・知っている」という層とは別に、運転中にウインカーへ勝手に手が伸びるような、場面に紐づいた記憶の層があります。agent に欠けていたのは、こっちでした

今回、これを実装してみました。

新しい記憶のスタイルと、それを運用に載せるときに押さえておきたいハーネスのパターンを作りましたので、シェアします。

6,622 文字



見出し ▾



下書きを保存しました

表面だけを再現しても、おそらく本来の価値は出ません。

どんな記憶のスタイルなのか。なぜ CLAUDE.md に書き足すだけでは届かないのか。どう組み込み、どう抑制するのか。そして前回書いた「マクロとミクロ」の構造の中で、これがどの隙間を埋めるのか。

✓ 実際のソースコードと動画解説

20260816 操作記憶 | Notion

動画

witty-lumber-ae4.notion.site



上記リンクからどうぞ

出典を入力

Claude Code の「自作ツールを使ってくれない問題」を toolhint で解決する

Claude Code を使い込むほど、自作の CLI や、チームの中でしか通じない造語が増えていきます。ところが、それらの説明を docs に書いておいても、agent は読まれてほしい瞬間にそこへたどり着かないまま作業を進めてしまいます。

この「書いてあるのに届かない」壁に対して作ったのが、今回紹介する Claude Code に入れる plugin、toolhint です。

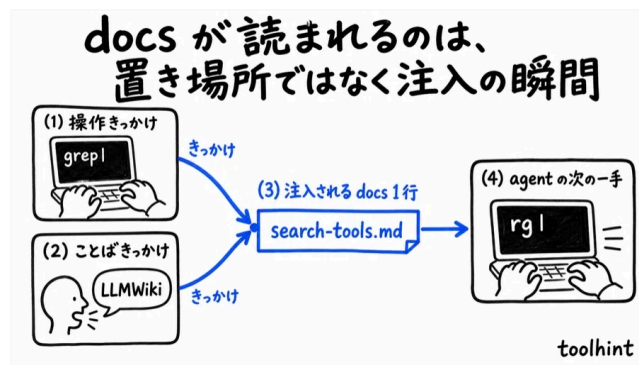
agent がコマンドを打とうとした直前に反応する操作きっかけと、ユーザーの発言に反応することばきっかけ、この 2 つに docs を紐づけて、届いてほしい瞬間に context へ注入します。

hook から知識を差し入れるやり方が足したのは、その注入を 1 件ず

下書きを保存しました

り中で使われてきました。toolhint から誰でも足し引きできるように

したことで、agent の記憶体系に「操作記憶」という置き場を 1 つ増やす部品です。



操作からでも、ことばからでも、届くものは同じです。docs のありかを指す 1 行だけが context に入り、grep をやめて rg を選ぶかどうかは agent 自身が決めます。

この記事で伝えたい できるようになること

- CLAUDE.md を太らせずに、規約や自作ツールの使い方を「必要な瞬間だけ」agent に思い出させられます
- 「zuroku deploy して」のような、チームでしか通じない頼み方でも、agent が正しい作業場へたどり着くようになります
- 作ったルールは repo にコミットしてチームへ配れて、合わないものは手元で止められます
- hook が壊れても作業は止まらない設計 (fail-open) なので、導入 2 コマンド + ルール 1 本から気軽に試せます

grep を打つ直前に docs の 1 行が入る

agent が `grep -r foo .` と打って検索しようとした瞬間、コマンドが走る前に 1 行だけ context に入ります。入るのは docs の中身ではなく、その置き場所です。

※ `grep` は、ファイルの中身から文字列を探すときに広く使われてきた検索コマンドです。

出典を入力

[toolhint] この操作の docs: ~/.claude/toolhint/docs/search-tools.md

下書きを保存しました

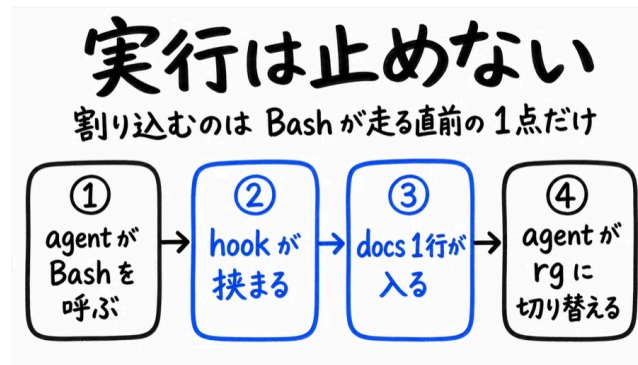


ト書き保存

公開に進む

前に反応するルールで、直いたのは Claude Code の hook です。読むかどうかが決めるのは agent の側です。

指し示された docs の中身は「grep より rg (ripgrep) を使う」という 1 枚です。agent はそれを読んで、同じ検索を rg で打ち直します。



割り込みは Bash が走る直前の 1 点だけです。hook が落ちてでも exit 0 が返るので、注入が失敗しても grep はそのまま走ります。

ルールの実体は 3 行です。

```
tool: Bash
match: \bgrep\b
doc: docs/search-tools.md
```

これに docs を 1 枚添えれば動きます。本体を入れるのは `claude plugin marketplace add` と `claude plugin install` の 2 コマンドで、最初のルールも 1 コマンドで置けます。

```
toolhint add --layer global --id use-rg --tool Bash --match '\bgrep\b'
```

書いたあとに `toolhint test "grep -r foo ."` を打つと、その入力で何が入るかを state を変えずに確かめられます。

守られていない癖を CLAUDE.md に足すか toolhint に登録するかは、ここで線が引けます。いつ読んでも効く規約は CLAUDE.md に、grep のようにその操作の瞬間だけ思い出せばいいものは toolhint のルールに置きます。

下書きを保存しました

ことばに反応させて、造語からマイクロハーネスへ渡す

チームの中では「zuroku deploy して」で話が通じますが、agent はその名前に反応する plugin も skill も持っていないので、近そうな操作を勝手に推測しに行きます。zuroku はチームの中でだけ通じる名前で、本当に渡したい先は、ライティング用や YouTube 企画用のように目的を絞って作り込んだ作業場 (マイクロハーネス) です。発言そのものを見て docs を注入するのが、toolhint のもう 1 つの側、ことばきっかけです。

ルールに event: prompt を書き、反応させたい語を keywords に並べます。区切りは読点でもカンマでもよく、並べたうちの 1 つが発言に含まれれば発火します。登録の確かめは toolhint test --event prompt "zuroku deploy して" で、実際には注入せずに結果だけ見られます。

```
id: zuroku-deploy
event: prompt
keywords: zuroku, ズロック
doc: docs/zuroku.md
```

僕はここに foyer を組み合わせています。foyer は、登録した作業場へ日本語の依頼をそのまま渡す窓口の CLI です。母艦の agent が「llm-wiki」と言われたときに「それは向こうの作業場なので foyer で渡す」と書いた docs が注入されれば、窓口を経由する判断がその場で入ります。

この組み合わせは、ルール 1 本で書けます。反応させたい語に llm-wiki と、大文字まじりの書き方も一緒に並べておくだけです。

```
id: llm-wiki-route
event: prompt
keywords: llm-wiki, LLMWiki
doc: docs/llm-wiki-route.md
```

指している docs には、llm-wiki が母艦とは別のマイクロハーネスであることと、依頼文は foyer の ask でそのまま向こうへ渡すことの 2 点を書いておきます。

登録する語は絞ったほうがうまくいきます。持ち場になるのは、一般語でなく、しかも手元の plugin や skill をたどっても agent が関連を引けない名前だけです。自作 CLI のようにコマンド名がそのまま打たれるものは、操作側のマイクロハーネスに、agent の側に登録する必要はありません。

下書きを保存しました

注入の効き方は層とマージ順で決まる

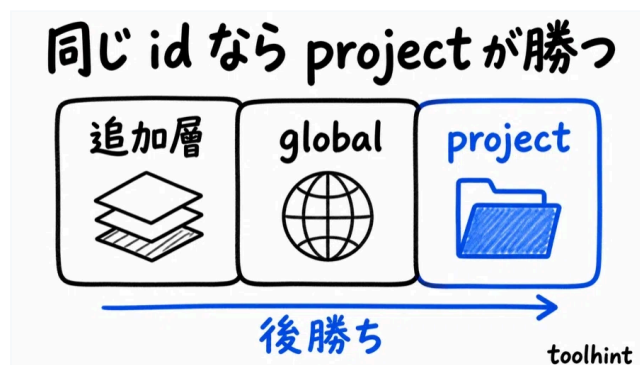
注入が思ったとおりに来ないときの切り分け

注入が思ったとおりに来ないときの切り分け	
症状	見るところ
docs が一度も来ない	管理画面に「未配線」が出ていないかを見ます。hook が Claude Code 側に登録されていないと、ルールを作っても注入は起きません
最初の 1 回しか来ない	既定は同じセッションに 1 回だけです。毎回渡したいルールには <code>every: always</code> を書きます
覚えのないルールが来る	同じ id のルールが別の置き場にないかを探します。読み込みは後から読んだものが残ります

キャプションを入力

3 行目に出てくる置き場が層です。

ルールと docs を入れておくディレクトリのことで、配られたものを入れる追加層 (~/.claude/toolhint/layers.d)、自分のマシン全体で使う global (~/.claude/toolhint/)、repo にコミットして PR でレビューできる project (<dir>/.claude/toolhint/) の 3 つがあります。toolhint はこの 3 つを決まった順に読み、その順番をマージ順と呼びます。



キャプションを入力

配られたルールを黙らせたいときは、この後勝ちを使います。手元の層に同じ id のルールを置いて `enabled: "false"` と書けば、配られた側のファイルには触らずに止まります。

置き場が 3 つあると身構えますが、1 人で使い始めるあいだは global だけで足りる。project と追加層が要るのは、ルールを人に渡す段になってからです。どのルールがいつ入ったかは 127.0.0.1:4517 の管理画面に注入の記録として残るので、増えすぎたと感じたらそこから間引けます。

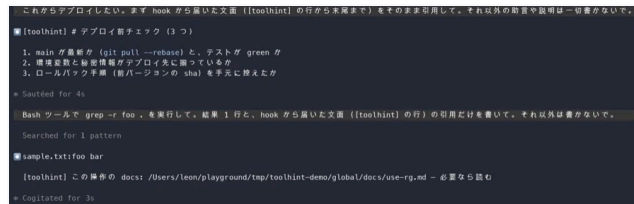
壊れたときの挙動も先に見ておくと導入を決めやすくなります。hook は設定が壊れていても終了コード 0 で抜けるので、ツールの実行もユーザーの発言も止まりません。止まるのは注入だけで、この振る舞いを fail-open と呼びま

下書きを保存しました

実際のセッションで注入を確かめる

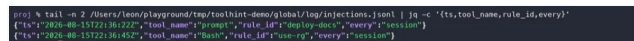
デモ用の Claude Code セッションで、まず「デプロイしたい」と打ちました。発言に含まれた語をことばきっかけが拾い、デプロイ前に見る 3 項目がそのまま context に入ります。続けて `grep -r foo` を頼むと、Bash が走り出す手前で操作きっかけが動き、rg の docs へ案内する 1 行が届きました。

画面に文面が出ているのは、「届いた文をそのまま貼って」と頼んだからです。ふだんは頼まないで、agent は何も言わずに docs を開いて作業を続けます。



キャプションを入力

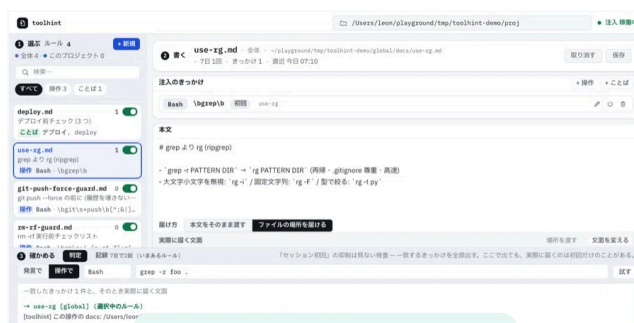
引用は agent 任せなので、証拠は別にとってあります。注入のたびに hook 自身が `log/injections.jsonl` へ 1 行足すので、agent が何を喋ったかとは関係なく履歴が残ります。実演で増えたのは 2 行で、`deploy-docs` と `use-rg` が、どちらもセッション初回の発火として並んでいました。



hook が書いた注入ログを末尾から 2 行だけ出し、jq で 4 つの項目に絞ったところです。時刻、反応した相手が発言か Bash か、ルールの id、初回だけの発火なのかが 1 行ずつ残ります。

同じ手順は、Python が 3.9 のままの別の Mac でも通しました。Python の標準ライブラリしか使っていないので、先に入れるものはありません。

ここまでは CLI での話ですが、toolhint には管理画面も同梱されていて、node manager/server.mjs を叩いてブラウザで 127.0.0.1:4517 を開くと立ち上がります。Node の標準機能しか使っていないので npm install もいりません。docs を軸にルールを並べ、届く文面を直し、注入を試し、いつどのルールが入ったかの記録まで同じ画面で扱えます。



下書きを保存しました

管理画面です。左に登録済みのルールが並び、中央で選んだ docs の中身と届け方をその場で直せます。下の欄で `grep` の入力を試したところで、`use-rg` が一致し、実際に届く 1 行までその場で確かめられます。

ルールは 1 本ずつは小さくても、増えるほど何がどこで動くのか見えなくなります。hooks で似た仕組みを組むこと自体は前からできて、今回それを目で見ながら足したり直したりできるようにした、というのが作者の説明です。並べて確かめられて、あとから記録で振り返れることが、hook を自分で書いていた頃との違いです。

一致条件の正規表現を自分で書くのが面倒なら、頼んで書かせられます。

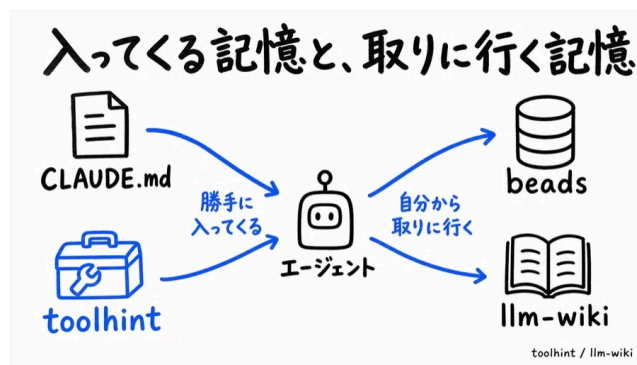
```
toolhint add --ai "git push --force を打つ前に、履歴を壊さない代替と注意点を出して" --lay
```

この 1 文だけで、ルールと docs の本文が起案されて global 層に追記され、直後の dry-run で狙いどおり発火しました。管理画面の「ことばで頼む」の欄に普通の文で書いても同じです。提案が返るまではデモの計測で約 10 秒、rules.yaml が書き換わるのは「この内容で保存」ボタンを押した後です。

操作記憶と、取りに行く記憶を分ける

toolhint が持つのは、操作とことばの 2 つに反応して、こちらが呼ばなくても注入される記憶です。これとは別に、必要になった時点で自分から引きに行きたい知識、とりあえず残しておくだけの知識があります。

後者を受けているのが llm-wiki です。会社・人・コミュニティといった外の世界について気づいたことを markdown のファイルにためていくエージェント用の知識ベースで、書くのも読むのもエージェント、人間はファイルを開きません。



エージェントに知識が届く向きは 2 つあります。CLAUDE.md と toolhint はエージェントが求めていなくても届き、beads や llm-wiki はエージェントが自分で引いたときだけ届きます。器を選ぶとは、この向きを選ぶことです。

下書きを保存しました

引きに行く側は、窓口の CLI である foyer 越しに自然文で頼みます。foyer ask llm-wiki "org/anthropic の記録を全部出して" と投げれば、repo の中の Claude が司書として探して返します。僕の並べ方では、llm-wiki は CLAUDE.md や beads のように形の決まった記憶より一段下の置き場で、判断の材料になる前の気づきを受け止める場所です。

分け方が要るのは、届く側の器ほど詰め込みたくなるからです。ルールは増えるほど何がいつ効いたのか追えなくなるので、入れるのは本当に要るものだけにしたほうがいいと思っていますし、まだ作りたてで不具合もありうる、というのは先に断っておきます。

置き場所ではなく、届く瞬間の側に登録する

この記事で見てきたことを貫く軸は 1 つです。docs が効くかどうかは、「どこに書いたか」ではなく「いつ届いたか」で決まります。

思い出させたいことは、置き場所ではなく、届く瞬間の側に登録します。toolhint がやったのは、その瞬間を操作とことばの 2 つで持てるようにしたこと。この 2 つがあるだけで、読まれるかどうかを祈るしかなかった決まりごとが、届く瞬間を指定できるものになります。

だから、増やす前に決めるべきは「どの瞬間に思い出してほしいか」のほうです。それが言えるものだけを toolhint 側へ回し、いつ読んでも役に立つものは常設の置き場に残します。試すなら、`claude plugin marketplace add AI-Driven-R-D-Dept/toolhint` と `claude plugin install toolhint@toolhint` の 2 つが次の一歩になります。まずは 1 本だけ登録して、その注入が本当に届いてほしい瞬間に来るかを見てみてください。

下書きを保存しました

